SOAP vs REST: Comparativa entre Servicios Web

Análisis colaborativo de dos arquitecturas para la comunicación entre sistemas: el protocolo formal SOAP y el estilo arquitectónico REST, aplicados al caso del Sistema de Gestión para Bancos de Ropa y Artículos de Segunda Mano.

¿Qué son SOAP y REST?

SOAP

Simple Object Access Protocol

Protocolo de comunicación estandarizado que define reglas estrictas para el intercambio de información estructurada entre aplicaciones, usando mensajes en formato XML llamados **envelopes (sobres)**. Su contrato se describe formalmente mediante un archivo **WSDL**.

Componentes: Envelope, Header, Body, Fault, WSDL, UDDI

Protocolos de transporte: HTTP, SMTP, TCP, JMS

Formato de datos: XML únicamente

Características clave: contrato formal, independiente del transporte, soporta transacciones ACID y WS-Security

REST

Representational State Transfer

Estilo arquitectónico (no un protocolo) que trata cada elemento del sistema como un **recurso** identificado por una URI, manipulado mediante los verbos estándar de HTTP. No mantiene estado entre peticiones (**stateless**).

Componentes: Recursos (URIs), métodos HTTP (GET, POST, PUT, DELETE), representaciones, headers HTTP

Protocolo de transporte: exclusivamente HTTP / HTTPS

Formato de datos: JSON, XML, texto plano, HTML

Características clave: sin estado, interfaz uniforme, cacheable, múltiples formatos

Semejanzas

• Ambos permiten la comunicación entre aplicaciones distintas a través de una red.

• Ambos pueden funcionar sobre el protocolo HTTP.

• Ambos se usan para construir e integrar servicios web (APIs).

• Ambos permiten el intercambio de datos entre plataformas y lenguajes diferentes.

• Ambos pueden integrarse con mecanismos de autenticación y seguridad.

Diferencias principales

• SOAP es un **protocolo** con reglas rígidas; REST es un **estilo arquitectónico** flexible.

• SOAP solo usa XML; REST admite JSON, XML y otros formatos.

• SOAP requiere un contrato formal (WSDL); REST no exige un estándar obligatorio.

• SOAP puede ser stateful; REST es siempre stateless.

• SOAP es más pesado y lento; REST es más ligero y rápido.

Ventajas y desventajas de cada arquitectura

SOAP

VENTAJAS

Seguridad robusta con WS-Security · Soporta transacciones ACID · Estándar formal y documentado (WSDL) · Independiente del protocolo de transporte

DESVENTAJAS

Mensajes XML pesados y verbosos · Mayor consumo de ancho de banda · Implementación más compleja · Menor velocidad de respuesta

REST

VENTAJAS

Ligero y rápido · Fácil de implementar y consumir · Cacheable, mejora el rendimiento · Amplia compatibilidad con JSON y apps móviles/web

DESVENTAJAS

Menor seguridad nativa (depende de HTTPS/OAuth) · Sin estándar formal estricto · Sin soporte nativo de transacciones complejas

Tabla comparativa

CRITERIO	SOAP	REST
Arquitectura	Protocolo con estándar formal (XML + WSDL)	Estilo arquitectónico basado en recursos
Protocolo de comunicación	HTTP, SMTP, TCP, JMS	Exclusivamente HTTP / HTTPS
Formato de intercambio de datos	XML únicamente	JSON, XML, texto plano, HTML
Flexibilidad	Baja — estructura rígida y contrato fijo	Alta — múltiples formatos y métodos
Rendimiento	Menor, mensajes pesados	Mayor, mensajes ligeros
Seguridad	WS-Security, robusta y estandarizada	Depende de HTTPS, OAuth o tokens
Facilidad de implementación	Compleja, requiere definir WSDL	Sencilla e intuitiva
Escalabilidad	Limitada por el peso de los mensajes	Alta, el carácter stateless facilita escalar
Compatibilidad	Buena con sistemas legados y empresariales	Excelente con aplicaciones web y móviles modernas
Casos de aplicación	Banca, pagos electrónicos, ERP, gobierno	APIs públicas, apps móviles, redes sociales, microservicios

Tres ejemplos reales de SOAP

PayPal SOAP API (NVP/SOAP clásico)

Tipo: SOAP / XML con WSDL

Funcionamiento: procesa pagos y transferencias mediante mensajes XML estructurados, con operaciones definidas formalmente en su contrato de servicio.

Problema que resuelve: integración segura y auditable del procesamiento de pagos entre comercios y la plataforma.

Información: datos de transacciones, montos, cuentas y confirmaciones de pago.

Servicios web del SAT (México)

Tipo: SOAP

Funcionamiento: el SAT expone servicios SOAP para el timbrado y validación de Comprobantes Fiscales Digitales (CFDI).

Problema que resuelve: validación y timbrado fiscal seguro de facturas electrónicas a nivel nacional.

Información: XML de facturas (CFDI), sellos digitales y folios fiscales.

Amadeus / Sabre (GDS de viajes)

Tipo: SOAP

Funcionamiento: los sistemas globales de distribución (GDS) usan SOAP para que agencias de viaje consulten y reserven vuelos y hoteles en tiempo real.

Problema que resuelve: interoperabilidad entre aerolíneas, hoteles y agencias con alta exigencia de seguridad transaccional.

Información: disponibilidad de vuelos, tarifas, reservas y datos de pasajeros.

Tres ejemplos reales de REST

API de X (Twitter)

Tipo: REST / JSON

Funcionamiento: expone endpoints REST para publicar y consultar tweets y perfiles, devolviendo respuestas en JSON.

Problema que resuelve: permite a desarrolladores integrar funciones de la red social en aplicaciones externas.

Información: tweets, datos de usuarios y métricas de interacción en JSON.

Google Maps Platform API

Tipo: REST

Funcionamiento: servicios REST para geocodificación y cálculo de rutas, consumidos mediante URLs con parámetros y respuestas en JSON.

Problema que resuelve: integrar funciones de mapas y ubicación en aplicaciones de terceros.

Información: coordenadas, direcciones y rutas en formato JSON.

Spotify Web API

Tipo: REST

Funcionamiento: API REST para buscar canciones y álbumes, controlar la reproducción y obtener recomendaciones.

Problema que resuelve: dar acceso programático al catálogo musical y a las funciones del usuario.

Información: metadatos de canciones, artistas, playlists y estado de reproducción en JSON.

CONCLUSIONES DEL EQUIPO

Tras comparar ambas arquitecturas, el equipo concluye que SOAP resulta más adecuado en entornos corporativos donde la seguridad formal, las transacciones garantizadas (ACID) y la interoperabilidad con sistemas legados son prioritarias, como la banca, el gobierno y los sistemas ERP. REST, en cambio, es la

opción natural para aplicaciones modernas, web y móviles, gracias a su ligereza, rapidez y facilidad de consumo con JSON.

Para nuestro proyecto — el Sistema de Gestión para Bancos de Ropa y Artículos de Segunda Mano (ODS 12) — **REST es la arquitectura más adecuada**: no requiere transacciones bancarias complejas ni los contratos formales de SOAP, y permite priorizar la velocidad de desarrollo y una integración sencilla entre el backend en Laravel y el frontend del sistema.
